Experiment No: 3			
Course Name:	Web Programming Laboratory (216U01L405)	**Semester:**	**IV**
Date of Performance:	**06/02/2026**	**DIV/ Batch No:**	**B2**
Student Name:	**Ashwera Hasan**	**Roll No:**	**16010124107**

Title: Develop a JavaScript program that dynamically manipulates HTML elements in the DOM

Aim of the Experiment:

- To create a web application that provides an interactive user experience by dynamically updating content and structure of the webpage in response to user actions.
- To enhance the understanding of DOM manipulation using JavaScript, allowing greater flexibility in web design.

Objectives for the Experiment:

1. Implement Event Listeners:
    a. Attach event listeners to user interface elements (e.g., buttons) to trigger JavaScript functions.
2. Manipulate DOM Elements:
    a. Use JavaScript to dynamically add, remove, and modify HTML elements based on user interactions.
    b. Change attributes, styles, and inner content of elements.
3. Create Functions:
    a. Develop specific JavaScript functions that handle different types of interactions:
        i. Adding new elements (e.g., a new item to a list).
        ii. Removing existing elements (e.g., deleting a list item).
        iii. Modifying properties (e.g., changing text color or visibility).
4. Optimize User Experience:
    a. Implement features like form validation and feedback messages based on user inputs.
5. Ensure Compatibility:
    a. Test and ensure that the JavaScript works across different browsers.

COs to be achieved:

Design dynamic web pages using various HTML tags.
Use CSS to prepare the layout of web pages.
Apply JavaScript for validation in client-side programming.

Books/ Journals/ Websites references:

1. Students should write

Theory:

Students should write about CSS3 (citation needed for content taken (IEEE or APA7 style))

CSS3 is the latest version of the cascading style sheets after CSS2. It has introduced many more elements and functionalities that make the webpages richer, more interactive, and visually appealing. Furthermore, it reduces reliance on scripting.

Like CSS2, CSS3's focus remains on the segregation of document content from its presentation. This separation is very helpful in accentuating the maintainability, flexibility, and accessibility of code.

Citation:
Sushmitha, V. (2025, June 19). What is CSS3? | BrowserStack. BrowserStack. https://www.browserstack.com/guide/what-is-css3

Problem statement: "College Event & Student Services Web Portal" A college wants to develop a basic HTML5-based web portal to showcase events, registrations, information, and services for students. Each task contributes one page or feature of this portal. Enhanced the portal using CSS3 and Java script .

Task 1: JavaScript Initialization & DOM Access

Page: index.html (Home Page)

Task:

Write JavaScript to dynamically display:

- Current date
- Welcome message using student name

JS Concepts:

- <script> / external JS
- DOM access (getElementById, querySelector)

Task 2: Variables, Data Types & Operators

Page: event-details.html

Task:

Calculate and display:

- Total number of available seats
- Remaining seats after registrations

JS Concepts:

- let, const
- Number, string, boolean
- Arithmetic and comparison operators

Task 3: Control Flow – Event Category Selection

Page: event-details.html

Task:

Use if–else or switch to:

- Display event category (Technical / Cultural / Sports)
- Show different messages based on category

JS Concepts:

- Control flow statements

Task 4: Arrays – Event List Management

Page: clubs.html

Task:

Store event names in an array and:

- Display them dynamically as a list
- Highlight featured events

JS Concepts:

- Arrays
- Looping (for, forEach)

Task 5: Functions – Reusable Logic

Page: clubs.html

Task:

Create reusable functions to:

- Display events
- Count total events

JS Concepts:

- Function declaration
- Arrow functions

Task 6: DOM Manipulation – Show/Hide Content

Page: about-dept.html

Task:

Add buttons to:

- Show/Hide Vision and Mission sections
- Toggle HoD message

JS Concepts:

- DOM manipulation

- style.display

Task 7: Event Handling – User Interaction

Page: navigation.html

Task:

Add JavaScript to:

- Highlight menu item on hover
- Display clicked page name

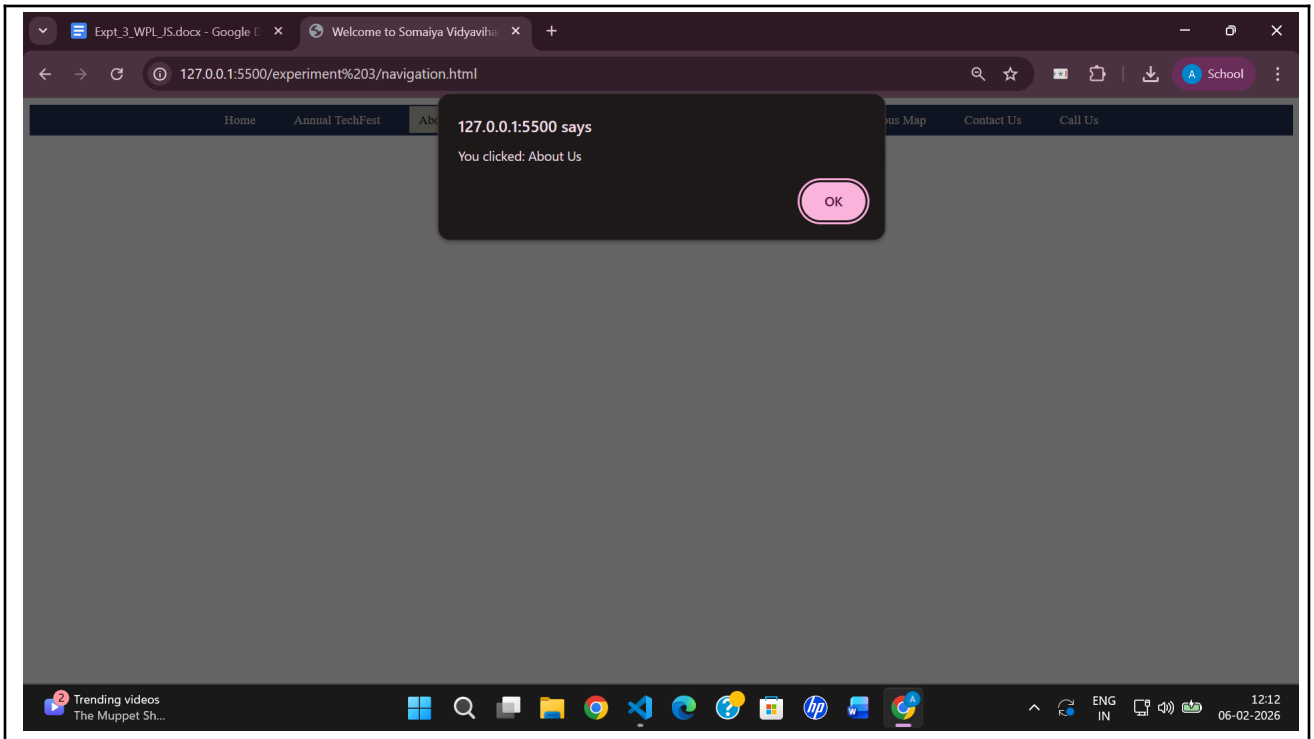
JS Concepts:

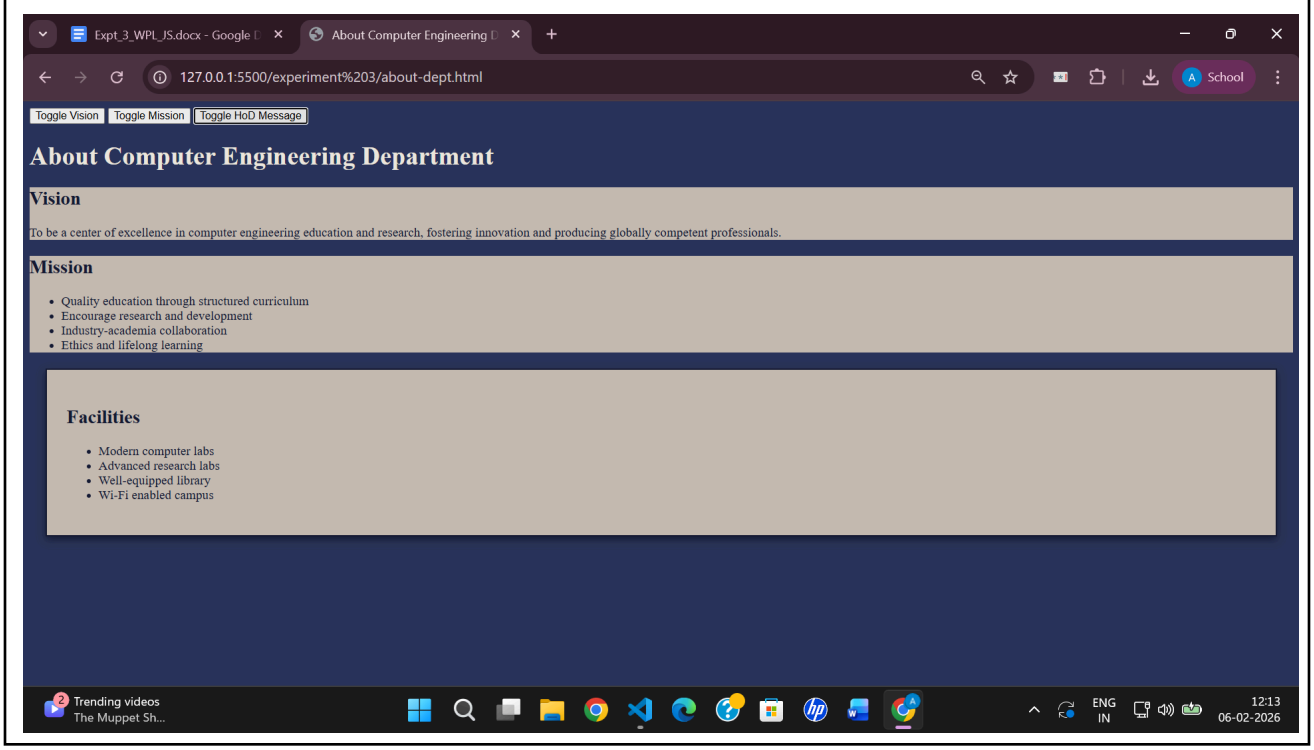
- addEventListener
- Mouse events

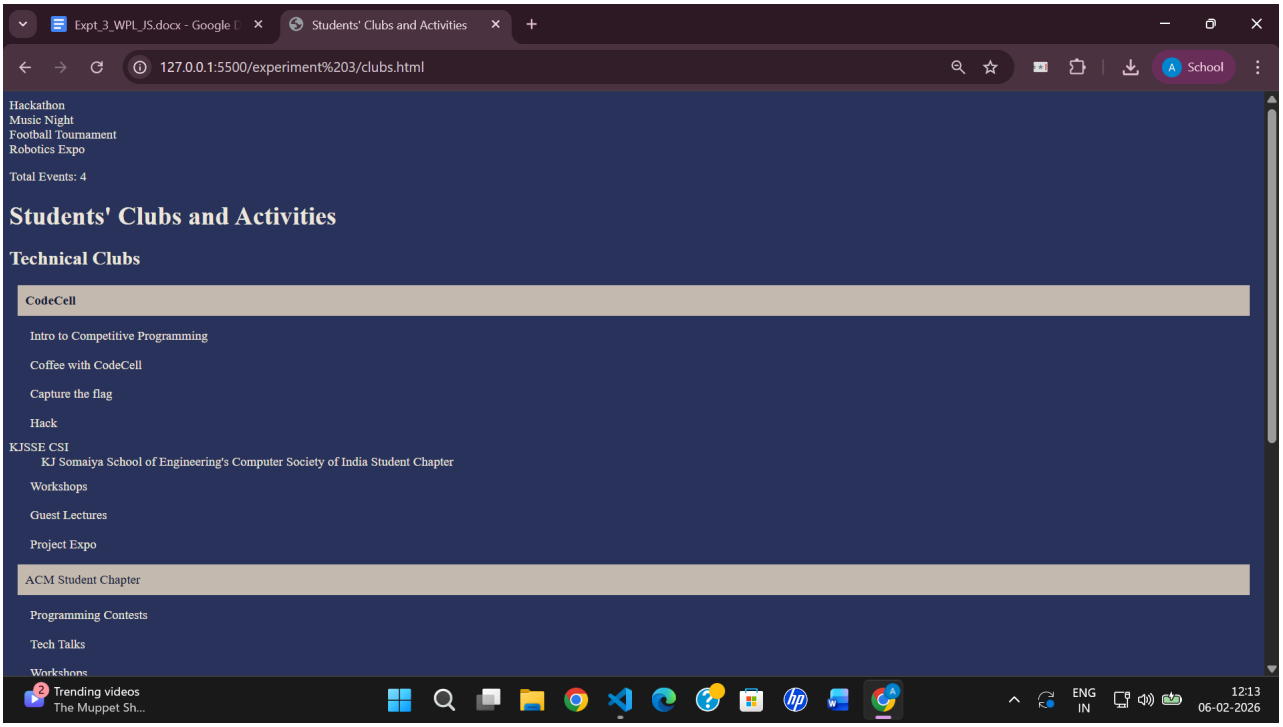
Code:

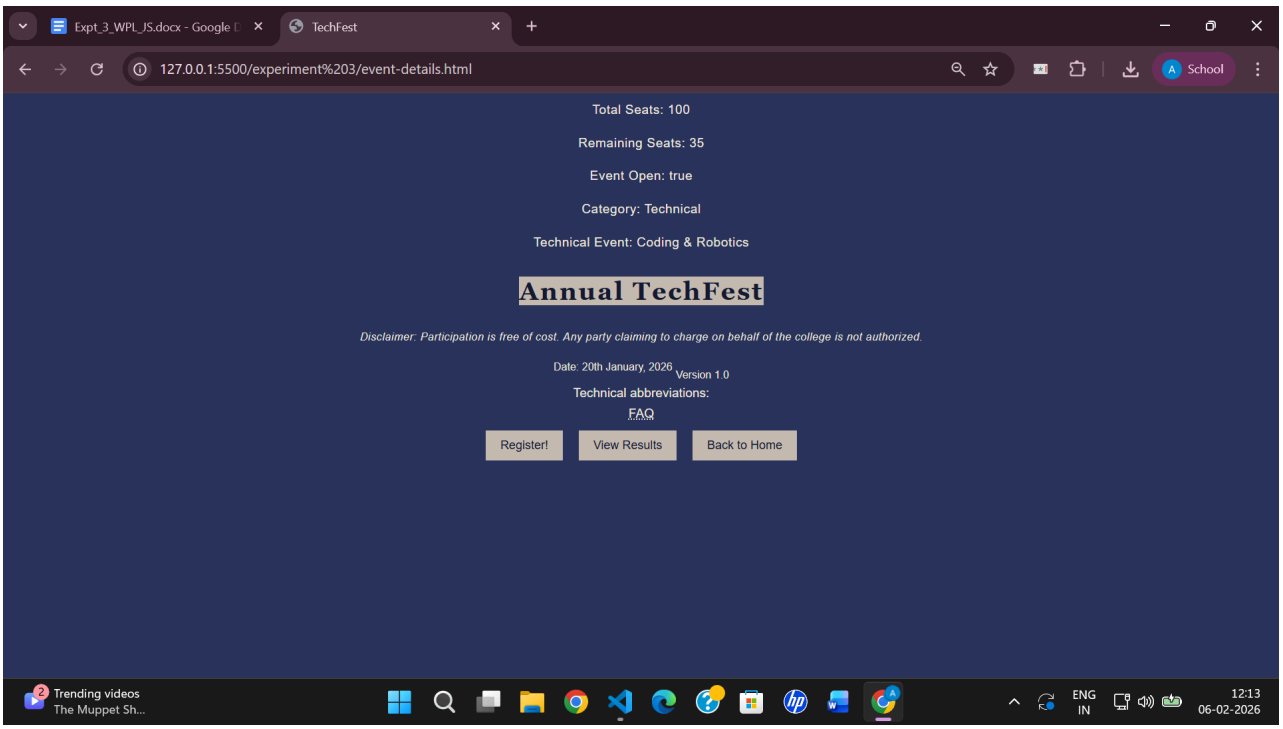
https://github.com/ashwera/webProgrammingLab/tree/main/experiment%203

Output / Screenshot:


Expt_3_WPL_JS.docx - Google
Welcome to Somaiya Vidyavih
127.0.0.1:5500/experiment%203/navigation.html
School
Home
Annual TechFest
Contact Us
Call Us
127.0.0.1:5500 says
You clicked: About Us
OK
Trending videos
The Muppet Sh...
ENG
IN
12:12
06-02-2026


Expt_3_WPL_JS.docx - Google
About Computer Engineering
127.0.0.1:5500/experiment%203/about-dept.html
School
Toggle Vision
Toggle Mission
Toggle HoD Message
About Computer Engineering Department
Vision
To be a center of excellence in computer engineering education and research, fostering innovation and producing globally competent professionals.
Mission
Quality education through structured curriculum
Encourage research and development
Industry-academia collaboration
Ethics and lifelong learning
Facilities
Modern computer labs
Advanced research labs
Well-equipped library
Wi-Fi enabled campus
Trending videos
The Muppet Sh...
ENG
IN
12:13
06-02-2026





Post Lab Subjective/Objective type Questions:

1. What are the differences between event bubbling and event capturing? Provide a code example that demonstrates both.
   Event bubbling and event capturing describe how events propagate in the DOM.
   1. In event bubbling, the event starts from the target element and moves upward to its parent elements, meaning the inner element handles the event first and then the outer elements; this is the default behavior in JavaScript.
   2. In event capturing, the event starts from the outermost parent and moves downward to the target element, so the parent handles the event before the child, and this behavior must be explicitly enabled by passing true as the third argument to addEventListener. For example, when clicking a button inside a div, capturing will trigger the div's event handler first and then the button's, while bubbling will trigger the button's handler first and then the div's.

Code:

```
const parent = document.getElementById("parent");
const child = document.getElementById("child");

// Bubbling
parent.addEventListener("click", () => {
  console.log("Parent - Bubbling");
});
child.addEventListener("click", () => {
  console.log("Child - Bubbling");
});

// Capturing
parent.addEventListener(
  "click",
  () => {
    console.log("Parent - Capturing");
  },
  true
);
child.addEventListener(
  "click",
  () => {
    console.log("Child - Capturing");
  },
  true
```

```
);
```

2. Describe how you could optimize DOM manipulation for better performance.
   DOM manipulation should be optimized because frequent or inefficient updates are slow. Access the DOM as little as possible and store elements in variables instead of querying repeatedly. Make multiple changes together rather than one by one to reduce reflows and repaints. Use document fragments or build HTML strings before inserting them into the DOM. Prefer event delegation instead of adding many event listeners. Minimize layout thrashing by avoiding repeated reads and writes to layout-dependent properties.

3. What are closures, and how can they be utilized in event handling in JavaScript?
   A closure is created when a function remembers and accesses variables from its outer scope even after that outer function has finished executing. Closures allow data to be preserved between function calls. In event handling, closures are used to maintain state or pass private data to event handlers. For example, an event listener can access variables defined outside it, such as counters, configuration values, or IDs, without using global variables.

4. Explain the concept of delegation in event handling and provide a scenario where it might be beneficial
   Event delegation is a technique where a single event listener is attached to a parent element instead of multiple child elements. It works by using event bubbling to detect events from child elements at the parent level. This improves performance, reduces memory usage, and simplifies code. Event delegation is beneficial when handling events for many similar elements, such as click events on list items in a dynamic list, where items may be added or removed frequently.

Conclusion and Discussion:

The experiment was able to demonstrate the effect and importance of Javascript integration with webpages to implement dynamic webpages.

Feedback:
Riya: The website looks cool but the alignment, padding, and margin is not proper.
Mohammad: You could make the colours better.
Bharghav: Functionalities could be more complex.